

Agentic Payments: The Next Evolution in the Payments Value Chain

by Piyush Mattoo, Haresh Nandwani, and Pablo Villegas | on 13 NOV 2025 | in [Amazon API Gateway](#), [Amazon Bedrock](#), [Amazon Bedrock Agents](#), [Amazon DynamoDB](#), [Amazon Simple Storage Service \(S3\)](#), [Artificial Intelligence](#), [AWS Marketplace](#), [Best Practices](#), [Customer Solutions](#), [Financial Services](#), [Generative AI](#), [Industries](#), [Intermediate \(200\)](#), [Learning Levels](#), [Thought Leadership](#) | [Permalink](#) | [Share](#)

The payments industry stands at a transformative inflection point. Traditional payment systems built on rigid rules, manual processes, and legacy infrastructure find it difficult to navigate the evolving landscape of modern commerce. Today's payment ecosystem demands solutions that handle increasing complexity while delivering the seamless, contactless, and autonomous experiences customers expect.

Financial institutions have long leveraged artificial intelligence (AI) and machine learning (ML) across the payments value chain. These early successes have built the foundation and confidence for more ambitious AI implementations, in the evolving field of Generative AI. AI agents and advances in Large Language Models (LLMs), along with the rollout of the Model Context Protocol (MCP) open standard, have enabled firms to introduce "Agentic Payments".

Agentic Payments deploy autonomous AI agents that learn, adapt, and make real-time decisions across the entire payments value chain. These systems address three critical challenges: transaction volume scalability, regulatory compliance automation, and personalized customer experiences.

Agentic payments leverage AI to create autonomous, intelligent payment systems capable of learning, adapting, and deciding in real-time across the entire cards and payments value chain. This blog post will explore:

1. Key use-cases for agentic payments, focusing on common challenges where intelligent agents outperform traditional rule-based systems and manual decision processes
2. A detailed examination of the Cognitive Payments Director (CPD) agent, which uses Agentic AI to implement payment routing for Payment Service Providers (PSPs)
3. Practical considerations for implementing agentic payment systems and realizing their transformative potential

Key agentic payments use cases

1. **Cognitive Payments Director:** As businesses expand, payments routing needs to address multiple variables and factors, including country-specific regulations, local payment methods, FX conversion costs, and country-specific fraud patterns. An agentic payment system tackles this challenge by creating a sophisticated routing intelligence that considers and evaluates multiple factors. Agents will understand the intricate relationships between payment gateways, local payment methods, currency requirements and regulatory frameworks, fee structures, and conversion rates in real-time, making sub-second decisions to optimize each transaction's route based upon business outcomes instead of complex and time-consuming static rules.
2. **FX Liquidity Management:** For Payment Service Providers handling multiple currency flows, liquidity management is key to serving their customers and revenue line. Agents could become experts in specific payment routes and currency pairs aligned to business drivers as opposed to fixed rules or patterns. These agents

optimize liquidity flows, considering factors like market dynamics, historical patterns, and settlement conditions. This is useful for industries such as online travel agencies, where understanding unique settlement time requirements is crucial. Agents will adapt to market conditions, ensuring optimal liquidity management while maintaining customer service levels.

3. **Cross-Border FX Trading:** In today's cross-border payment landscape, spread optimization is crucial for profitability. AI agents specialized in currency trading and business operations provide an effective solution by monitoring markets, analyzing patterns, and executing trades at optimal times. These systems balance competitive rates with settlement needs, enabling treasury operations to maximize yields while maintaining service quality. This intelligent approach helps businesses offer attractive pricing while protecting their margins in international transactions.
4. **Risk and Fraud Management:** In an era of increasing financial fraud, traditional rule-based systems are no longer sufficient. The agentic payment system introduces a multi-layered approach to risk management. Specialized AI agents focus on different aspects of fraud prevention – from Authorized Push Payment (APP) fraud to Card Not Present (CNP) transactions. These agents will monitor card network behavior, analyze payment scheme patterns, and track regulatory compliance. These agents collaborate with a centralized orchestration AI agent that makes real-time decisions about transaction legitimacy. This helps businesses maintain high security standards while minimizing false positives that impact legitimate customer transactions.
5. **Payments repair:** AI agents specialized in payment schemes and cross-border regulations will reduce manual intervention in multi-party transactions. These agents will access and consolidate information from various system fields, ensuring compliance with specific requirements like SWIFT, SEPA Direct Debits, or Faster Payments. For example, they will adhere to South African payment regulations or format addresses correctly for different schemes. By leveraging LLMs to interpret payment rules and local regulations, these agents will include required details such as payment reasons. This approach will help minimize back-office workload in resolving failed payments, enhancing efficiency in global transaction processing.
6. **E-commerce payments:** LLMs are poised to revolutionize online shopping by acting as personal shoppers, capable of finding and paying for items based on user preferences and budgets. For instance, an AI agent will plan and book a complete holiday package, including accommodations, activities, and dining, tailored to individual tastes. However, implementing secure payment processing within these systems remains a challenge. To realize this potential, integration with existing PCI-compliant tokenization systems is necessary, addressing issues of payment authorization and fraud liability across the transaction chain. This AI-driven approach also opens new avenues for targeted marketing, as LLMs can refine search results and offers, ushering in a new era of personalized e-commerce experiences.
7. **Accounts Receivable (AR) / Accounts Payable (AP) matching:** AI agents transform the complex task of reconciliation in large organizations where traditional AR/AP matching tools fall short. By understanding each customer's general ledger and chart of accounts, specialized agents (for example, fees associated with certain types of transactions, receivable invoices broken down into multiple payers) process multiple data sources from invoices and bank statements to ERP system data. These agents will work to identify matching patterns, process company-specific payment behaviors, and handle exceptions intelligently
8. **Payment dispute management:** AI agents streamline payment dispute resolution by automating key processes and enhancing decision-making. These systems categorize disputes, gather evidence, and track deadlines while analyzing patterns in fraud and customer behavior. By processing documentation across multiple languages and formats, agents assess case validity and recommend resolutions based on historical data and precedents. They

manage stakeholder communications, ensure regulatory compliance, and provide valuable insights for dispute prevention. This automated approach reduces manual processing time while improving consistency and accuracy in dispute resolution, allowing organizations to handle cases more efficiently while maintaining compliance standards.

Agentic AI use-case deep dive – Cognitive Payments Director (CPD)

Marketplaces face complex challenges in managing transactions between buyers and sellers across different currencies, payment methods, and regulatory requirements. Consider a scenario where a UK buyer uses a GBP debit card to purchase from an Italian seller who prefers EUR payments via a specific Italian-based payment provider. Traditional approaches either ignore optimization opportunities or struggle with operational complexity.

The Cognitive Payments Director is an Agentic AI-based orchestration system that streamlines this process by utilizing multiple specialized AI agents. These specialized agents work together to automate complex payment flows, reduce processing costs, and ensure regulatory compliance, transforming a static rules-based process into an automated and efficient system.

This intelligent system will enable organizations such as marketplaces to scale operations across different jurisdictions while adapting to changing market conditions, assessing multiple PSPs and regulatory requirements.

Role of AI Agents in implementing CPD

Agent 1. Financial controller agent – The responsibility of this agent is to understand existing contracts and conditions with Payment Service Providers (PSPs) and conditions the PSPs have agreed to provide and to understand the differences in KPIs subject to the need to settle Buyer and Creator. This agent will use an LLM pre-trained to identify specific legal details in a PSP contract, such as payment terms and financial figures, and can compare different versions of a contract. The KPIs are classified into transaction performance, risk and fraud, and revenue. The output is a weighted view of which PSP will meet either the buyer's inbound payment selection or the PSP to meet the creator outbound settlement contract.

Agent 2. Payment conditions controller agent – This agent analyses payment method availability and contract terms across PSPs, considering:

- Payment method geographic restrictions and local preferences
- Settlement timing requirements (real-time vs next-day vs T+2)
- Transaction limits and daily caps per payment method
- Fee structures, including interchange, scheme fees, and FX spreads
- Volume-based pricing tiers and monthly commitment requirements
- Cross-border payment capabilities and regulatory restrictions

Agent 3. Legal controller agent – The responsibilities of this agent include ensuring compliance with regulations for intermediary organizations such as marketplaces, including specific requirements like mandatory escrow accounts for marketplace transactions in the United Arab Emirates (UAE). Additionally, the agent must understand the regulatory implications affecting liquidity and treasury positions while considering organizational cash requirements

for committed payments, such as accounts receivable and accounts payable. Based on the buyer and seller situation, such as a seller in Argentina preferring USD payments, the agent provides a response detailing the optimal payment methods to process the payment as requested by the decision maker agent.

Agent 4. PSP watch observer agent – The PSP Watch Observer Agent monitors and evaluates Payment Service Providers' operational performance and financial stability. It generates comprehensive health ratings for each PSP by analyzing key performance indicators, including authorization rates, decline rates, chargeback frequencies, refund patterns, fraud occurrences, transaction processing efficiency, and platform availability. To ensure accurate assessment, the agent collects and processes these metrics from multiple data sources, incorporating both direct PSP reporting and independent third-party verification systems.

Agent 5. Decision maker agent – The responsibility of this agent is to take inputs from all subagents described above and decide on the best PSP to initiate the payment and learn from these decisions to improve future routing decisions. The Decision Maker agent will take inputs from the Financial Controller, Legal Controller, and Watch Observer agents to decide on the most appropriate PSP to use for an incoming payment request.

These agents will operate within their defined scope, accessing additional datasets through APIs to retrieve supplementary information, including market information, for a detailed view of the implications.

What sets this solution apart is its continuous learning capability. The agents analyze transaction patterns, PSP performance, key metrics (for example, failure rates) and market conditions to refine their decision-making processes, resulting in improved success rates and reduced payment failures. This adaptability ensures that organizations maintain optimal payment operations even as market conditions develop, regulatory requirements change, and new payment methods emerge.

The transformation from traditional rule-based systems to this intelligent, autonomous network of agents enables organizations to focus on growth and customer experience while the agentic system handles the complexities of payment operations, ensuring long-term operational efficiency and accuracy.

Amazon Bedrock multi-agent architecture for implementing CPD

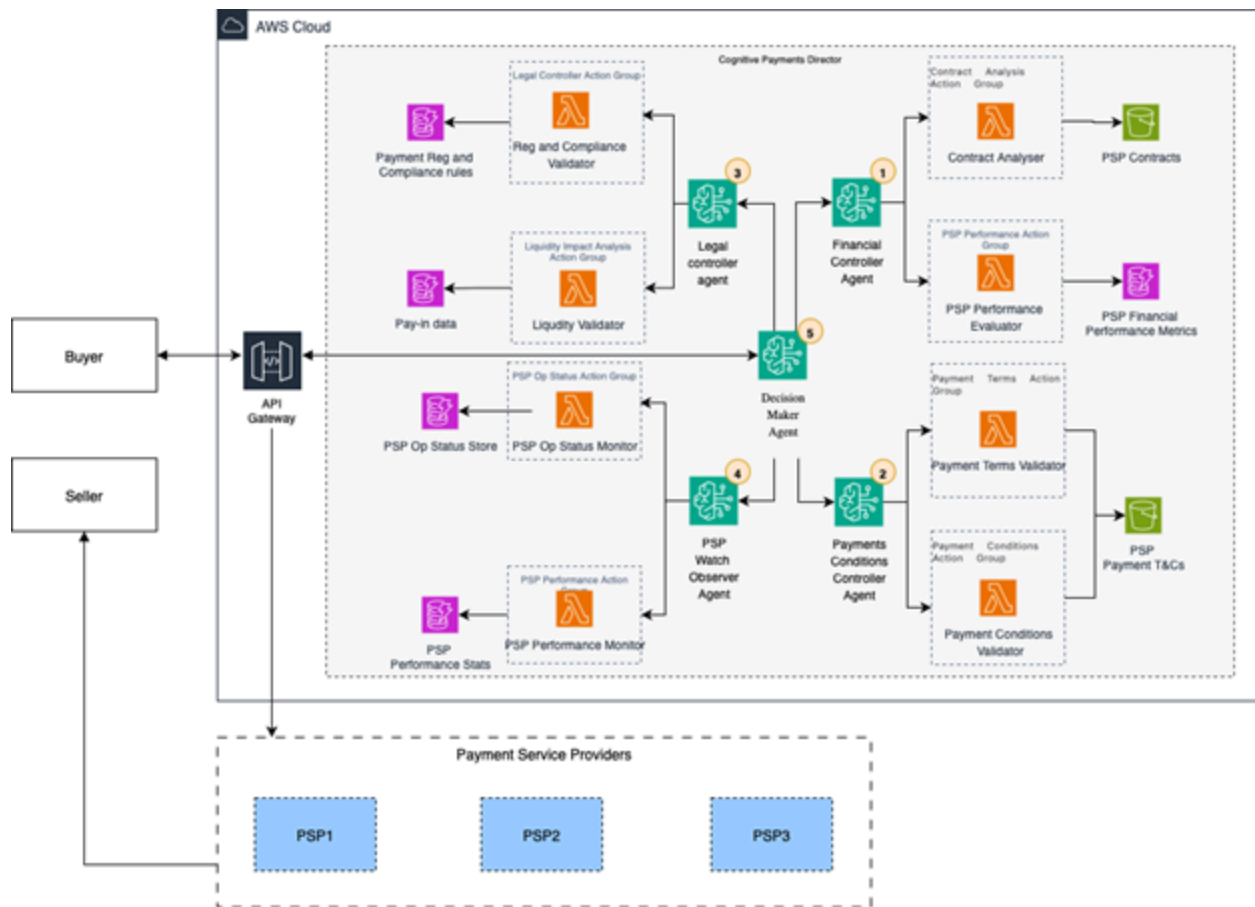


Figure 1 – Cognitive Payments Director multi-agent architecture

The Cognitive Payments Director (CPD) system uses AWS services to create a scalable, resilient, and intelligent payment routing system that optimizes for cost, success rates, and compliance while adapting to changing conditions in real time. Here, we use the [multi-agent collaboration pattern supported by Amazon Bedrock](#) agents to assemble a team of agents, each specializing in a specific domain, orchestrated by a centralized supervisor agent. The core architecture comprises:

- **Supervisor agent:**

- **Decision Maker Agent:** The central orchestrator that coordinates all other agents and makes the final routing decisions based on inputs from specialized agents. An AWS Bedrock foundation model (like the Claude or [Anthropic](#) model) powers the core intelligence of the Decision Maker agent. This provides the reasoning capabilities needed to synthesize inputs and make optimal routing decisions.
- In addition, [AWS Bedrock Agents](#) are used to refine the agent's capabilities, knowledge base, and action groups. Custom [AWS Lambda](#) functions implement the action groups that allow the Decision Maker agent to communicate with other specialized agents, process their response, apply business logic to make the final routing decision, and invoke the selected payment gateway.

- **Specialized collaborator agents:**

- **Financial Controller Agent** analyzes financial metrics through two action groups:
 - **Contract Analyzer:** An AWS Lambda function evaluates PSP contract terms and connects to the PSP Contacts database on [Amazon Simple Storage Service \(Amazon S3\)](#)

- PSP Performance Evaluator: an AWS Lambda function that assesses PSP performance and connects to PSP Financial Performance Metrics on an [Amazon DynamoDB](#) table.
- **Payment Conditions Controller Agent** evaluates payment conditions through:
 - Payment Terms Validator: an AWS Lambda function that validates payment terms
 - Payment Conditions Validator: an AWS Lambda function that validates payment conditions
 - Both connect to the PSP Payment Terms and Conditions (T&Cs) database
- **Legal Controller Agent** ensures regulatory compliance through:
 - Regulatory and Compliance Validator: an AWS Lambda function that validates transactions against Payment Regulatory and Compliance rules on an Amazon DynamoDB table.
 - Liquidity Validator: an AWS Lambda function that analyzes the liquidity impact of payment decisions
- **PSP Watch Observer Agent** monitors PSP status and performance through:
 - PSP Op Status Monitor: AWS Lambda functions that track operational status and store in the PSP Op Status Store
 - PSP Performance Monitor: AWS Lambda functions that track performance metrics and store in PSP Performance Stats

Setting up multi-agent collaboration for CPD using Amazon Bedrock

1. Navigate to the Amazon Bedrock service and ensure you have access to the relevant model you plan to use (for example, Amazon Nova Premier) for the agent.
2. Create the four collaborator agents using the existing agent builder workflow. Open the Amazon Bedrock console, select Agents in the left navigation panel, then choose Create Agent.
 - a. Create the Financial Controller Agent as shown in the screenshot below

Create agent ✕

Name

financial-controller-agent

Valid characters are a-z, A-Z, 0-9, _ (underscore) and - (hyphen). The name can have up to 100 characters.

Description - *optional*

The responsibility of this agent is to understand existing contracts and conditions for PSPs and differences in KPIs for each PSP

The description can have up to 200 characters.

Multi-agent collaboration

[Learn more about multi-agent collaboration](#)

☐ **Enable multi-agent collaboration**

If you have more than one agent, multi-agent collaboration allows this agent to associate the others as agent collaborators to orchestrate responses. You can change this later.

Cancel Create

Figure 2 – Amazon Bedrock – Create agent screen

b. In the Agent builder dialog box, choose to create and use a new service role, select Amazon Nova Premier as the model, and provide the following instructions for the agent

You are a financial controller agent that can analyze existing PSP contracts and conditions for PSPs and the difference in KPIs for each PSP. You will generate a score and rating for each PSP that will help decide which PSP is the best for routing the incoming payment request.

c. Next create the Contracts Analyzer action group as shown in the screenshot below

Create Action group

Action group details

Enter Action group name

Description - *optional*

Evaluates PSP contract terms and connects to PSP Contacts data

Valid characters are a-z, A-Z, 0-9, _ (underscore) and - (hyphen). This description can have up to 200 characters.

Action group type

Select what type of action group to create

☒ **Define with function details**
Specify functions and define parameters as JSON objects that will be associated to the action group invocation.

☐ **Define with API schemas**
Specify a Lambda or API Gateway and define a schema to specify the APIs that the agent can invoke to carry out its tasks.

Figure 3 – Amazon Bedrock – Create action group screen, Action group type

Select “Quick create a new Lambda function” option. This will create a Lambda function in your account. Add code to this Lambda function to access the PSP contracts data stored in an S3 bucket.

Action group invocation

Specify a Lambda function that will be invoked based on the action group identified by the Foundation model during orchestration.

Select how to define the Lambda function

☒ **Quick create a new Lambda function - recommended**
An Amazon Lambda function will be created in your account on your behalf. No further configurations are necessary.

☐ **Select an existing Lambda function**
Use an existing Lambda function for this action group.

☐ **Return control**
Agent responses in the test window will prompt the user for function details to generate a response. No further configurations are necessary.

Figure 4 – Amazon Bedrock – Create action group screen, Action group invocation

▼ Action group function 1 : analyse_co

Delete

Table

JSON Editor

A function specifies the business logic for this action group using the defined parameters. Create functions using the form builder or via JSON editor. Up to 3 functions can be created per action group.

Name

analyse_contracts

Valid characters are a-z, A-Z, 0-9, _ (underscore) and - (hyphen). This description can have up to 100 characters.

Description - optional

Perform analysis on contracts data stored in S3

Valid characters are a-z, A-Z, 0-9, _ (underscore) and - (hyphen). This description can have up to 1200 characters.

Enable confirmation of action group function - optional

Request confirmation before user invokes the action group's function. Enabling this may safeguard your end user from malicious prompt injection.

☐ Enabled
☒ Disabled

Parameters (0)

Parameters allow you do define object relationships within the action group function.

Name	Description	Type	Required	Delete
No parameters				

Add parameter

Figure 5 – Amazon Bedrock – Create action group screen, Action group function

d. Create the PSP Performance Evaluator action group as shown in the screenshot below

Create Action group

Action group details

Enter Action group name

psp_performance_evaluator

Description - optional

Access and evaluates PSP performance data

Valid characters are a-z, A-Z, 0-9, _ (underscore) and - (hyphen). This description can have up to 200 characters.

Action group type

Select what type of action group to create

☒ Define with function details
 Specify functions and define parameters as JSON objects that will be associated to the action group invocation.

☐ Define with API schemas
 Specify a Lambda or API Gateway and define a schema to specify the APIs that the agent can invoke to carry out its tasks.

Figure 6 – Amazon Bedrock – Create action group screen, Action group type

Select “Quick create a new Lambda function” option. This will create a Lambda function in your account. Add code to this Lambda function to access the PSP performance metrics stored in a DynamoDB table.

The screenshot shows the 'Action group function 1 : evaluate_p' configuration page in Amazon Bedrock. At the top, there are buttons for 'Delete', 'Table' (selected), and 'JSON Editor'. Below this, a text box explains that a function specifies business logic and that up to 3 functions can be created per action group. The 'Name' field contains 'evaluate_psp_performance'. The 'Description - optional' field contains 'AWS Lambda function assesses PSP performance and connects to PSP Financial Performance Metrics on Amazon DynamoDB'. Below the description, there are radio buttons for 'Enabled' and 'Disabled', with 'Disabled' selected. At the bottom, there is a 'Parameters (0)' section with a table header (Name, Description, Type, Required, Delete) and an 'Add parameter' button.

▼ **Action group function 1 : evaluate_p** Delete Table JSON Editor

A function specifies the business logic for this action group using the defined parameters. Create functions using the form builder or via JSON editor. Up to 3 functions can be created per action group.

Name

evaluate_psp_performance

Valid characters are a-z, A-Z, 0-9, _ (underscore) and - (hyphen). This description can have up to 100 characters.

Description - optional

AWS Lambda function assesses PSP performance and connects to PSP Financial Performance Metrics on Amazon DynamoDB

Valid characters are a-z, A-Z, 0-9, _ (underscore) and - (hyphen). This description can have up to 1200 characters.

Enable confirmation of action group function - optional

Request confirmation before user invokes the action group's function. Enabling this may safeguard your end user from malicious prompt injection.

☐ Enabled

☒ Disabled

Parameters (0)

Parameters allow you do define object relationships within the action group function.

Name	Description	Type	Required	Delete
No parameters				

Add parameter

Figure 7 – Amazon Bedrock – Create action group screen, Action group invocation

▼ Action group function 1 : evaluate_p

Delete

Table

JSON Editor

A function specifies the business logic for this action group using the defined parameters. Create functions using the form builder or via JSON editor. Up to 3 functions can be created per action group.

Name

evaluate_psp_performance

Valid characters are a-z, A-Z, 0-9, _ (underscore) and - (hyphen). This description can have up to 100 characters.

Description - optional

AWS Lambda function assesses PSP performance and connects to PSP Financial Performance Metrics on Amazon DynamoDB

Valid characters are a-z, A-Z, 0-9, _ (underscore) and - (hyphen). This description can have up to 1200 characters.

Enable confirmation of action group function - optional

Request confirmation before user invokes the action group's function. Enabling this may safeguard your end user from malicious prompt injection.

☐ Enabled
 ☒ Disabled

Parameters (0)

Parameters allow you to define object relationships within the action group function.

Name	Description	Type	Required	Delete
No parameters				

Add parameter

Figure 8 – Amazon Bedrock – Create action group screen, Action group function

3. Repeat the steps above to create the additional collaborator agents
 - a. Legal Controller Agent – Also create the Reg and Compliance and Liquidity Impact Analysis action groups
 - b. PSP Watch Observer Agent – Also create the PSP Op Status and PSP Performance action groups
 - c. Payment Conditions Controller Agent – Also create the Payment Terms and Payment Conditions action groups
4. Next create the supervisor Decision Maker agent and select the “multi-agent collaboration” option as shown in the screenshot below

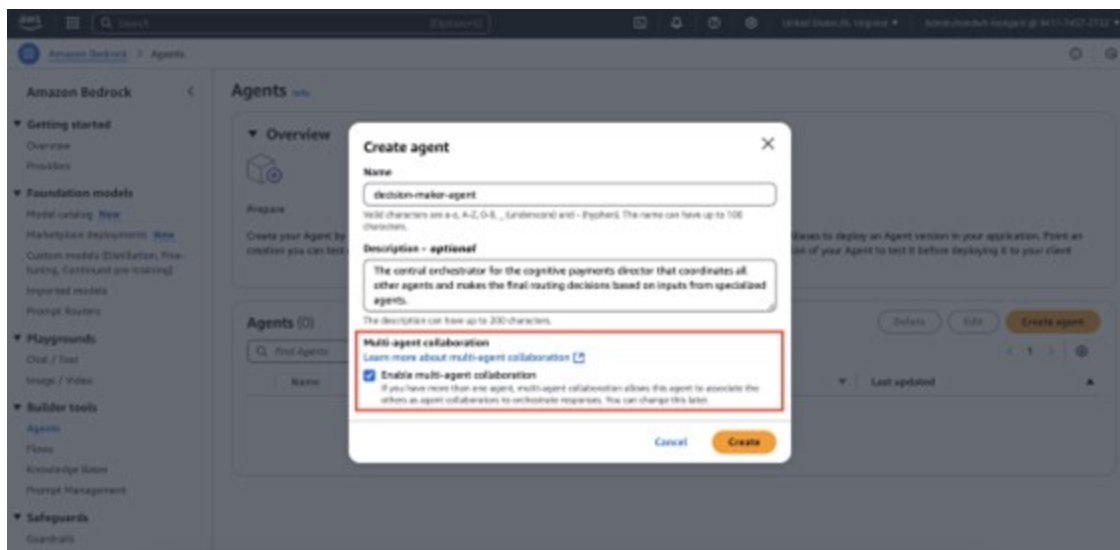


Figure 9 – Amazon Bedrock – Create agent screen

a. In the Agent builder screen select Amazon Nova Premier as the foundational model for the agent and provide the following instructions to the agent

Evaluate responses from the legal controller agent, PSP watch observer agent, payment conditions controller agent and the financial controller agent to identify the best possible payment route for the incoming payment request

Agent builder Info

Manual Assistant Test Prepare Save Save and exit

Agent details

Agent name

Valid characters are a-z, A-Z, 0-9, _ (underscore) and - (hyphen). The name can have up to 100 characters.

Agent description - optional

The description can have up to 200 characters.

Agent resource role
☐ Create and use a new service role
☒ Use an existing service role

Select model

Nova Premier 1.0 ⓘ ✎
 US Nova Premier

Instructions for the Agent
Provide clear and specific instructions for the task the Agent will perform. You can also provide certain style and tone.

Figure 10 – Amazon Bedrock – Agent builder screen

b. Setup multi-agent collaboration by using the “Add collaborator” feature

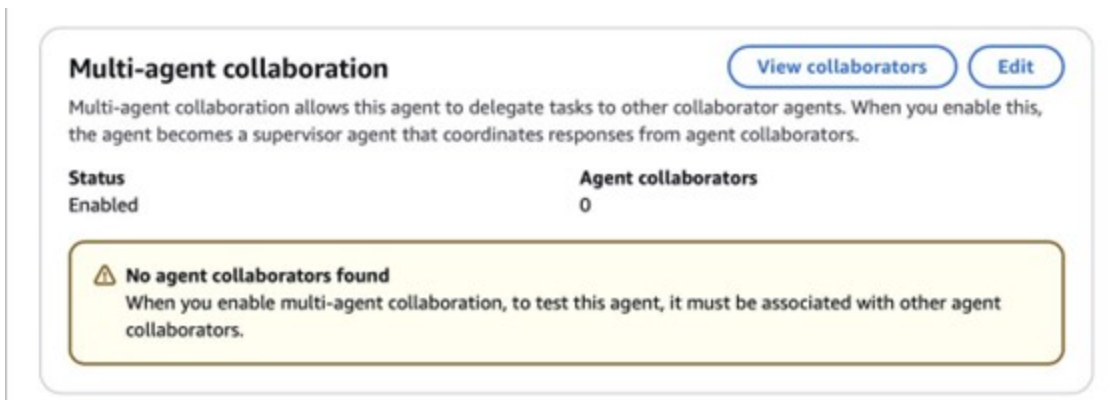


Figure 11 – Amazon Bedrock – Agent builder screen, Multi-agent collaboration section

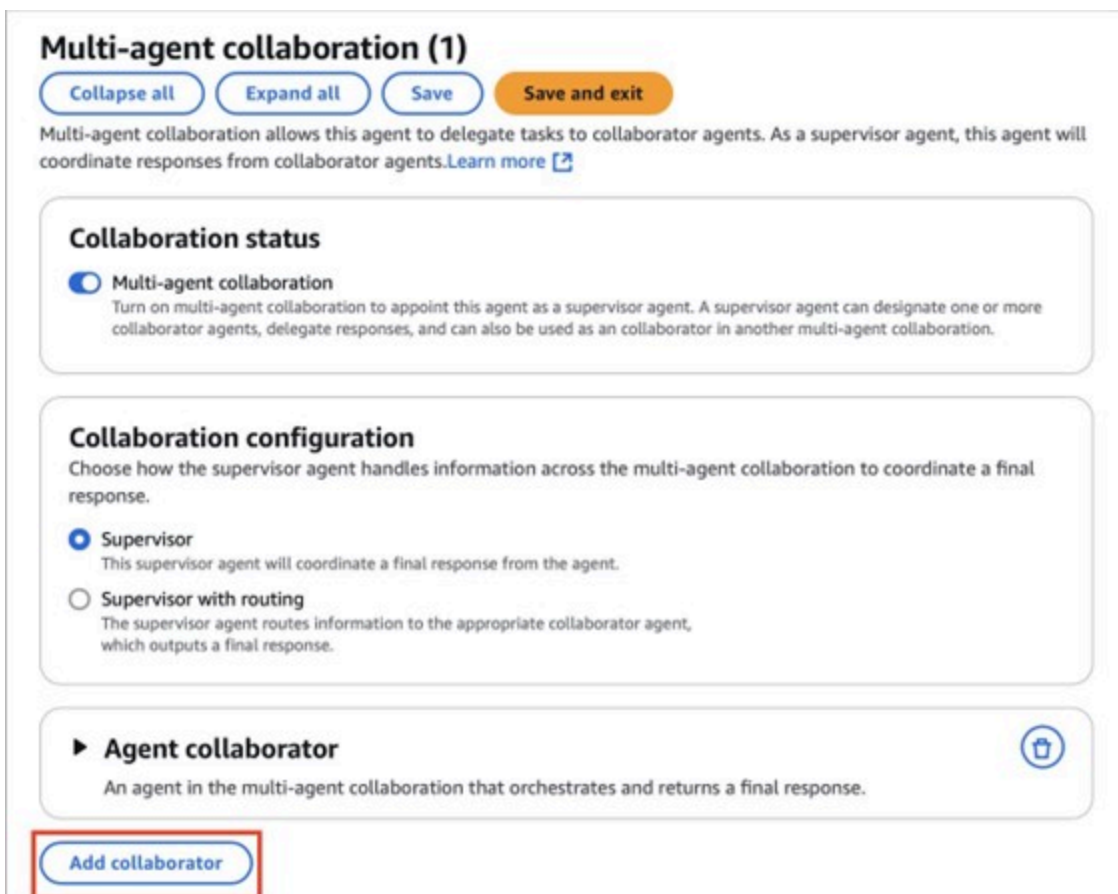


Figure 12 – Amazon Bedrock – Agent builder screen, Multi-agent collaboration, Agent collaborator

Add the Financial Controller agent as a collaborator and provide the following collaborator instructions

Invoke this agent to analyze existing contracts and conditions for each PSP and differences in KPIs. The agent responds with a rating and score for each PSP will help decide the best payment route.

▼ Agent collaborator : financial-controller-agent

Collaborator agent

financial-controller-agent ▼

Agent alias

financial_controller_V1 ▼

View

Collaborator name

An alternate name that appears only in collaboration instructions. It doesn't replace the original agent name.

financial_controller

Valid characters are a-z, A-Z, 0-9, _ (underscore), and - (hyphen). Collaborator name must be 1-50 characters.

Collaborator instruction

Describes the task that the agent performs within the multi-agent collaboration. We recommend that you provide clear, specific scenarios. You can also provide style and tone. Refer to the agent using its collaborator name.

and conditions for each PSP and difference in KPIs. The agent responds with a rating and score for each PSP to will help decide the best payment route

Collaboration instruction must be 40-2,000 characters.

☐ Enable conversation history sharing

When enabled, in future invocations, the supervisor agent can share context from previous responses with this collaborator agent.

Add collaborator

Figure 13 – Amazon Bedrock – Agent builder screen, Multi-agent collaboration, Agent collaborator

Add the legal controller agent as a collaborator agent and provide the following instructions

Invoke this agent to perform regulatory and compliance validation for the payment as well as identify the liquidity impact of executing the payment

▼ Agent collaborator : legal_controller_agent

Collaborator agent

legal_controller_agent ▼

Agent alias

legal_controller_v1 ▼

View

Collaborator name

An alternate name that appears only in collaboration instructions. It doesn't replace the original agent name.

legal_controller

Valid characters are a-z, A-Z, 0-9, _ (underscore), and - (hyphen). Collaborator name must be 1-50 characters.

Collaborator instruction

Describes the task that the agent performs within the multi-agent collaboration. We recommend that you provide clear, specific scenarios. You can also provide style and tone. Refer to the agent using its collaborator name.

You can invoke this agent to perform regulatory and compliance validation for the payment as well as identify liquidity impact of executing the payment

Collaboration instruction must be 40-2,000 characters.

☐ Enable conversation history sharing

When enabled, in future invocations, the supervisor agent can share context from previous responses with this collaborator agent.

Add collaborator

Add the payment conditions controller as a collaborator agent using the instructions above and provide the following instructions

Invoke this agent to analyze payment conditions for both pay-ins and pay-outs. The agent responds with a set of best routes based upon existing contracts and rules to process pay-ins and pay-outs.

Add the PSP Watch Observer agent as a collaborator agent using the instructions above and provide the following instructions

Invoke this agent to understand the status of PSP based on real-world data and metrics from external sources. The agent will respond with a status that will determine the best payment route.

c. Prepare the Decision Maker agent and create an alias. Integrate with your APIs hosted on AWS API Gateway.

Data and processing flow for CPD

1. The buyer starts a marketplace payment transaction through a web portal, mobile app, or [point of sale](#) (POS) system. The payment details are collected, like amount, payment method, customer information, and so on.
2. The organization's system sends the payment request to the [AWS API Gateway](#) instance. AWS API Gateway authenticates the request using [API keys](#) or [IAM roles](#). AWS API Gateway then invokes an AWS Lambda function to validate the transaction data, enrich the request with additional merchant context and format the request for the payment routing system.
3. The Decision Maker Agent receives the request and coordinates with specialized agents for analysis.
4. Each specialized agent performs its analysis using AWS Bedrock action groups. An AWS Lambda function facilitates agent interactions and implements business logic for routing decisions.
5. Action groups connect to relevant data stores to retrieve and analyze information. The specialized agents return their analyses to the Decision Maker Agent.
6. The Decision Maker Agent determines the optimal PSP routing based on all inputs

Conclusion

The future of payments is intelligent, autonomous, and adaptive. Agentic payments represent an incremental improvement and a fundamental reimagining of how payment systems operate. We interact with these payments in a complex global economy.

Organizations looking to implement an agentic payment system should consider a phased approach to experimentation:

- Identify high-value use cases with agreed KPIs attached to ROI potential
- Start with focused agent implementations that complement existing systems
- Build robust data pipelines to feed agent decision-making processes

- Implement comprehensive monitoring and feedback mechanisms
- Expand agent autonomy as confidence and capabilities grow

AWS provides the foundational infrastructure and services necessary to implement agentic payment systems at scale. Our global network and partners network, combined with advanced machine learning capabilities and robust security features, enable financial institutions to:

- Deploy sophisticated AI models for payment processing
- Scale operations across regions
- Maintain compliance with local and international regulations
- Ensure high availability and disaster recovery
- Protect sensitive financial data

As the financial services industry continues to evolve, agentic payments will become critical for organizations looking to maintain a competitive advantage and deliver superior customer experiences.

Contact an [AWS representative](#) to discuss how we can help you implement agentic payment systems at scale with AWS, or visit [AWS for Financial Services](#) to learn more.

Further reading

- [Introducing Strands Agents, an Open-Source AI Agents SDK](#)
- [Multi-agent collaboration capability with Amazon Bedrock](#)
- [Build a multi-tenant generative AI environment for your enterprise on...](#)
- [Bedrock AgentCore](#)

TAGS: [commerce](#), [Orchestration](#), [payments](#)



Piyush Mattoo

Piyush is a Senior Solution Architect for the Financial Services Data Provider segment at Amazon Web Services. He's a software technology leader with over a decade of experience building scalable and distributed software systems to enable business value using technology.



Haresh Nandwani

Haresh leads AWS global Resilience Test Engineering field community while serving as a Principal Solutions Architect in AWS UK's Financial Services team. With deep expertise in well-architected practices, he partners with organizations to transform their cloud resilience strategies, guiding them through the design, implementation, and operation of resilient systems on AWS.



Pablo Villegas

Pablo is the EMEA Payments Head at AWS, defining and executing strategy whilst driving transformative cloud solutions with payments providers. Pablo plays a key role in shaping industry priorities, steering innovation and operational resilience for global and local organizations, delivering results in complex industry challenges.